Assignment No:- 7.

Q.1.

```
psvm ( ){
    int[] arr = {5,2,3,2,4};
    int n=2;
    int target =fun (arr,0,n);
    sop (target);
}
fun (int arr[],int i, int n){
    if (i==arr.length){
        return-1;
    }
    if (n==arr[i]){
        return i;
    }
    return fun (arr,i+1,n);
}
```

2

```
fun (arr, 0, 2){
    if (i==arr.length)×
    if (n == arr[i]) 2==5×
    retun fun (arr, i+1, n);
```

```
fun (arr, 0+1, 2){
    if (i==arr.length)×
    if (n== arr[i]){
        2==2 ✓
        return i;
    }
```

Output

2

fun(0)

fun(1)

TC = $O(n)$

SC = $O(1)$

Q2) last occurrence.

length = 5

```
PSVM ( ) {
    int[] arr = {5, 2, 3, 2, 4};
    int n = 2;
    int target = fun (arr, n, 0);
    Sop (target);
}
```

$\boxed{3}$

```
fun (int[] arr, int n, int i) {
    if (i == arr.length) {
        return -1;
    }
    int right = fun (arr, n, i+1);
    if (right != -1) {
        return right;
    }
    if (arr[i] == n) {
        return i;
    }
    return -1;
}
```

```
fun (arr, n, i) {
    if (i == arr.length) X
    right = fun (arr, n, i+1);
    if (right != -1)
        return right;
}
```

```
fun (arr, n, 0+1) {
    if (i == arr.length) X
    right = fun (arr, n, i+1);
    if (right != -1)
        return right;
}
```

```
fun (arr, n, 0+1) {
    if (i == arr.length) X
    right = fun (arr, n, i+1);
    if (right != -1)
        return right;
}
```

```
fun (arr, n, 3) {
    if (i == arr.length) X
    right = fun (arr, n, i+1);
    if (right != 1) X
    if (arr[i] == n
        2 == 2 ?
    return i;
```

```
fun (arr, n, 4) {
    if (i == arr.length) X
    right = fun (arr, n, i+1);
    if (right != -1) X
    if (arr[i] == n)
        4 == 2 X
    return -1;
```

```
fun (arr, n, 5) {
    if (i == arr.length)
        5 == 5
        return -1;
}
```

Q3) Count occurence.

$t.c = \boxed{O(n)}$

$s.c = \boxed{O(1)}$

```
psvm() {
    int[] arr = {1,2,2,3,2};
    int target = 2;
    int ans = fun(arr, target, 0, 0);
    Sop(ans);
}
fun(int[] arr, int target, int count, int i){
    if (i == arr.length)
        return count;
    if (arr[i] == target)
        count++;
    return fun(arr, target, count, i+1);
}
```

output = $\boxed{3}$

① fun(arr, 2, 0, 0)

if (i == arr.length) ✗

0 == 2

if (arr[i] == target) ✗

$\boxed{0}$ count++;

return fun(arr, target, count, i+1);

fun(arr, 2, 0, 1)

if (i == arr.length) ✗

if (arr[i] == target) ✓

$\boxed{1}$ count++;

return fun(arr, target, count, i+1)

fun(arr, 2, 1, 2)

if (i == arr.length) ✗

if (arr[i] == target) ✓

$\boxed{2}$ count++;

fun(arr, target, count, i+1)

fun(count = 0, i = 1)

fun(arr, 2, 2, 3)

if (arr.length == i) ✗

if (arr[i] == target) ✗

count++;

return fun(arr, target, count, i+1)

fun(arr, 2, 2, 4)

if (arr.length == i) ✗

if (arr[i] == target) ✓

$\boxed{3}$ count++

return fun(arr, target, count, i+1)

fun(arr, 2, 3, 5)

if (arr.length == i)

return count;

Q4] if Array is sorted

```
psvm ()
    int arr[] = {1,2,3,4};
boolean ans = fun(arr, 0);
    sop(ans);
}
fun(arr[], i)
    if (arr.length == i)
        return true;
    if (arr[i] ≥ arr[i+1])
        return false;
    return fun(arr, i+1);
}
```

```
fun(arr, 0)
    if (arr.length == i)
        4-1 == 0 ✗
    if (arr[i] ≥ arr[i+1])
        return false ✗
    return fun(arr, i+1)
fun(arr, 1)
    if (arr.length == i) ✗
    if (arr[i] > arr[i+1]) ✗
    return fun(arr, i+1)
fun(arr, 2)
    if (  ) ✗
    if (arr[i] > arr[i+1]) ✗
    return fun(arr, i+1)
fun(arr, 3)
    if (arr.length-1 == i)
        return true;
}
}
```

tc = $\boxed{O(n)}$

sc = $\boxed{O(1)}$

output = $\boxed{\text{true}}$

Q5) All elements are even.

```
psvm ( ) {
    int arr[] = {2, 4, 6}
    bool ans = fun(arr, 0);
    sop (ans);
}
fun(arr, i ) {
    if (i == arr.length) {
        return true;
    }
    if (arr[i] % 2 != 0) {
        return false
    }
    return fun(arr, i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{\text{true}}$

```
fun(arr, 0) {
    if (i == arr.length) ✗
    if (arr[0] % 2 != 0) ✗
    return fun (arr, i+1);
}
fun(arr, 1) {
    if (i == length) ✗
    if (arr[1] % 2 != 0) ✗
    return fun(arr, i+1);
}
fun (arr, 2) {
    if (i == length) ✗
    if (arr[2] % 2 != 0) ✗
    return fun(arr, i+1);
}
fun(arr, 3) {
    if (i == length) ✓
        return true;
}
}
```

Q 5] element is negative

```
psvm ()
    int arr[] = {1, 2, -3, 4};
boolean ans = fun(arr, 0);
    sop(ans);
}
fun(arr, index){
    if (arr.length == i){
        return true;
    }
    if(arr[i] < 0){
        return false;
    }
    return fun(arr, i+1);
}
```

output = false

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

```
fun(arr, 0){
    if (arr.length == i)x
    if(arr[i] < 0) x
    return fun(arr, i+1);
}
fun(arr, 1){
    if (arr.length == i)x
    if (arr[i] < 0) x
    return fun(arr, i+1);
}
fun(arr, 2){
    if(arr.length == i)
    if(arr[i] < -3){
        -3 < -3 ✓
        return false
}
```

Q7] psvm( ){

```
int[] arr={3,9,1,7};
int max = arr[0];
int ans = fun(arr,max,0);
sop(ans);
}
fun(int arr[],int max, int i){
    if(arr.length == i){
        return max;
    }
    if (max < arr[i]){
        max = arr[i];
    }
    return fun(arr, max, i);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{9}$

```
fun (arr, max, 0){
    if(arr length == i)×
    if(max < arr[i])×
return fun (arr, max, i+1);
}
fun (arr, max, 1){
    if(arr.length == i)×
    if (max < arr[i]){
    [9] max = arr[i];
    }
return fun(arr, max, i+1);
}
fun (arr, max, 2){
    if (arr.length == i){
    if (max < arr[i])×
return fun (arr, max, i+1);
```

```
fun (arr, max, 3){
    if (arr.length == i)✓
        return max
}
```

Q8) psvm () {
int arr[] = {3, 9, 1, 7};
int mini = arr[0];
int ans = ifun (arr, mini, 0);
sop(ans);

}

fun(arr, mini, i) {
if(arr.length == i) {
return mini;

}
if (mini > arr[i]) {
mini = arr[i];

}
return fun(arr, mini, i+1);

}

fun(arr, mini, 0) {
if (length == i) x
if (mini > arr[i]) x
return fun(arr, mini, i+1);

}

fun(arr, mini 1) {
if(length == i) x
if(mini > arr[i] x
return fun (arr, mini, i+1);

}

fun(arr, mini 2) {
if(length == i) x
if(mini > arr[i] x
mini = arr[i];

}
return fun(arr, mini, i+1);

fun(arr, mini 3) {
if(length == i) {
return;

}

}

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{1}$

Q9) psvm() {

```
    int arr[] = {10,20,30,40,50}
    int sum = 0;
    int ans = fun(arr, sum, 0);
    sop(ans);
}
fun(arr, sum, i) {
    if(arr.length == i) x
    if(arr[i] % 2 == 1) {
        sum += arr[i];
    }
    return fun(arr, sum, i+1);
}
fun(arr, sum, 1) {
    if(arr.length == i) x
    if(arr[i] % 2 == 1) {
        sum += arr[i]
return fun(arr, sum, i+1);
}
fun(arr, sum, 2) {
    if(length == i) x
    if(arr[i] % 2 == 1) x
return fun(arr, sum, i+1);
}
fun(arr, sum, 3) {
    if(length == i) x
    if(arr[i] % 2 == 1) {
    60 sum += arr[i]
return fun(arr, sum, i+1);
fun(arr, sum, 4) {
    if(length == i) x
    if(arr[i] % 2 == 1) x
return fun(arr, sum, i+1);
fun(arr, sum, 5) {
    if(length == i)
return sum
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

Output: $\boxed{60}$

Q10)

```
psvm( ){
    int arr[] = {1,2,3,2,1};
    boolean ans = fun(arr, 0, arr.length-1);
    sop (ans).
}
fun (int[] arr, int first_i, int last_i){
        if (first_i == last_index){
            return true;
        }
        if (arr[first_i] != arr[last_i]){
            return false;
        }
        return fun (arr, first_i+1, last_i-1);
}
```

```
fun( arr, 0, 4){
    if( 0 == 4){ ✗
    if (arr[0] != arr[4]) ✗
    return fun (arr, 0+1, 4-1);
}
fun (arr, 1, 3){
    if (1 == 3) ✗
    if (arr[1] != arr[3]) ✗
    return fun(arr, 1+1, 3-1);
}
fun(arr, 2,2){
    if (2 == 2){
        return true;
    }
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{\text{true}}$

Q12)

```
psvm( ){
    int[] arr={1,2,3,3};
    boolean ans = fun(arr, 0);
    sop(ans);
}
fun(arr, i){
    if (arr.length-1==i){
        return true;
    }
    if (arr[i] >=arr[i+1]){
        return false;
    }
    return fun(arr, i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{\text{false}}$

```
fun(arr,0){
    if(arr.length==i)
    if(arr[i] >= arr[i+1])
return fun(arr, i+1);
}
fun(arr1){
    if(arr.length-1==i)
    if(arr[i] >= arr[i+1])
return fun(arr, i+1);
}
fun(arr2){
    if(length-1==i)
    if(arr[i] >= arr[i+1]){
        return false;
    }
}
```

Q.13).

```
psvm()?
 arr = {1,5,3,7,2}
 x = 3
 count = 0;
 int ans = fun(arr,x,count,0);
}
fun(arr, x, count, i)?
    if (length == i)?
        return count;
    }
    if (arr[i] > x)?
        count ++;
    }
    return fun(arr, x, count, i+1);
}
fun(arr, x=3, 0, 0)?
    if (length == i)x
    if (arr[i] > x)x
    return fun(arr, x, count, i+1);
}
fun(arr, x, 0, 1)?
    if(length == i)x
    if(arr[i] > x)?
        1 count ++;
    }
    return fun(arr, x, 1, i+1);
}
fun(arr, x, 1, 2)?
    if(length == i)x
    if (arr[i] > x)x
    return fun(arr, x, 1, i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{2}$

```
fun (arr, x, 1, 3)?
    if (length == i);
    if (arr[i] > x)x
    2 count ++;
    }
    return fun (arr, x, 2, i+1);
}
fun (arr, x, 2, 4)?
    if (length == i)x
    if (arr[i] > x)x
    return fun (arr, x, 2, 5);
}
fun (arr, x, 2, 5)?
    if (length == i)?
        return count;
    }
```

Q.14.

```
int arr = {1,2,3,4};
target = 3;
boolean ans = fun (arr, target, 0);
}
fun(arr, target, i){
    if (length == i){
        return false
    }
    if (arr[i] == target);
        return true;
    }
    return fun(arr, target, i+1);
}
fun(arr, target, 0){
    if(length == i)
    if(arr[i] == target)
    return fun (arr, target, i+1);
}
fun (arr, 3, 1){
    if(length == i)
    if( arr[i] == target)
    return fun (arr, target, i+1);
}
fun (arr, 3, 2){
    if (length == i)
    if (arr[i] == target){
        return true;
    }
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

Output = true

Q.15.

```
psvm() {
    int arr[] = {1,-2,3,-4,5};
    int sum = 0;
    int ans = fun(arr,sum,0);
}
fun(arr,sum,i) {
    if(i == length) {
        return sum;
    }
    if(arr[i] < 0) {
        sum += arr[i];
    }
    return fun(arr,sum,i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = -6

```
fun (arr, sum, 0) {
    if (i == length) x
    if (arr[i] < 0) x
    return fun(arr,sum,i+1);
}
fun(arr, 0, 1) {
    if(i == length) x
    if(arr[i] < 0) {
        [-2] sum += arr[i];
    }
    return fun(arr, sum, i+1)
}
fun (arr, -2, 2) {
    if(i == length) x
    if (arr[i] < 0) x
    return fun(arr, -2, 3);
}
fun (arr, -2, 3) {
    if(i == length) x
    if (arr[i] < 0) {
        [-6] sum += arr[i];
    }
    return fun(arr, sum, i+1);
}
fun (arr, -6, 4) {
    if (i == length) x
    if (arr[i] < 0) x
    return fun (arr, sum, i+1);
}
fun (arr, -6, 5) {
    if(i == length)
        return sum;
}
}
```

Q18).

```
psvm ()
    int[] arr = {5, 2, 8, 1, 3};
    int mini = arr[0];
    int mini_index = 0;
    int ans = fun(arr, mini, mini_index, 0);
    sop(ans);
}
fun (int[] arr, int mini, int mini_index, int index)
    if (arr.length == index)
        return mini_index;
    }
    if (mini > arr[index])
        mini = arr[index];
        mini_index = index;
    }
    return fun (arr, mini, mini_index, index+1);
```

Q.17)

```
psvm( ){
    int[] arr={2,4,5,6,9};
    int k=2;
    ~~int count=0;~~
    int count=0;
    int ans=fun(arr,k,count,0);
}
fun(arr,k,count,i){
    if(arr.length==i){
        return count;
    }
    if(arr[i]%2==0){
        count++;
    }
return fun(arr,k,count,i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

Output = $\boxed{3}$

```
fun(arr,k,count,0){
    if(length==i)x
    if(arr[i]%k==k){
        [1] count++;
    }
return fun(arr,k,count,i+1);
fun(arr,k,i,1){
    if(length==i)x
    if(arr[i]%k==0){
        [2] count++;
    }
return fun(arr,k,count,i+1);
fun(arr,k,2,2){
    if(length==i)x
    if(arr[i]%k==0)%
return fun(arr,k,count,i+1)
}
```

```
fun(arr,k,2,3){
    if(length==i)x
    if(arr[i]%k==0){
        [3] count++;
    }
return fun(arr,k,3,i+1);
}
fun(arr,k,3,4){
    if(length==i)x
    if(arr[i]%k==0)x
}
return fun(arr,k,3,i+1);
}
fun(arr,k,3,5){
    if(length==i);
        return count; [3]
}
}
```

8].

```
psvm(){
    int[] arr = {1,2,3,4};
    boolean ans = fun(arr, 0);
    sop(ans);
}
fun(arr[], index){
    if(arr.length == index){
        return true;
    }
    for(int i=0; i< arr.length ; i++){
        if(arr[i] == arr[index] && index != i){
            return false;
        }
    }
    return fun(arr, index+1);
}
```

Q(19).

```
psvm() {
    int arr[] = {12, 34, 5};
    int sum = 0;
    int ans = fun(arr, sum, 0);
    sop (ans);
}

fun (arr[], sum, index) {
    if (i == length) {
        return sum;
    }
    int n = arr[i];
    while (n != 0) {
        sum += n%10;
        n /= 10;
    }
    return fun(arr, sum, i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{15}$

```
fun(arr, sum, 0) {
    if(i == length) x
    int n = arr[i]; 12
    while (n != 0) {
        3 sum += n%10;
        n /= 10;
    }
    return fun (arr, sum, i+1);
}

fun(arr, 3, 1) {
    if(i == length) x
    int n = arr[i] 34
    while (n != 0) {
        10 sum += n%10;
        n /= 10;
    }
    return fun(arr, sum, i+1);
}

fun(arr, sum, 2) {
    if(i == length) x
    int n = arr[i]; 5
    while (n != 0) {
        15 sum += n%10;
        n /= 10;
    }
    return fun(arr, sum, i+1);
}

fun (arr, sum, 3) {
    if(i == length) x
    ~~int n = arr[i]~~
    return sum;
}
}
```

Q 20].

```
psum ( ){
    int arr[] = {9,7,5,2};
  boolean ans = fun(arr ,0);
    sop(ans);
}
fun (arr , i){
    if(i ==length-1){
        return true;
    }
    if (arr[i] < arr[i+1]){
        return false;
    }
    return fun(arr, i+1);
}
```

TC = $\boxed{O(n)}$

SC = $\boxed{O(1)}$

output = $\boxed{\text{True}}$

```
fun(arr,0){
    if(i==length-1)
  if(arr[i]<arr[i+1])
return fun(arr, i+1);
}
fun(arr, 1){
    if(i==length-1)
    if(arr[i] < arr[i+1])
return fun(arr, i+1);
}
fun(arr,2){
    if(i==length)
    if(arr[i]<arr[i+1])
  return fun(arr, i+1);
}
```

```
fun(arr, 3){
    if(i==length-1)
        return true;
    }
}
```

~~Assignment No: 08.~~

Q1] psvm();

```
int[] arr = {10, 5, 8, 20, 15};
int max_element = arr[0];
for(int i=0; i< arr.length; i++)?
        if(arr[i] > max_element)?
                max_element = arr[i];
}       }
int second_element = arr[0];
int ans = fun (arr, max_element, second_element, 0);
sop (ans);

}
fun( int arr[], int max_element, int second_element, int index)?
        if (index == arr.length)?
                return second_element);
        }

        if (second_element < arr[index] && arr[index] != max_element)
                second_element = arr[index];
        }
return fun (arr, max_element second_element, index+1);
}
```